

Integración Backend (C++) / Frontend (Python)

Esta guía describe la estructura esperada del output del backend tomando de referencia al archivo `output_modelo.json` que debe generar el motor de C++. El frontend (escrito en Python) leerá este archivo para animar la simulación de manera automática.

Flujo entre backend y frontend

1. **Pre-cálculo:** El backend en **C++** se encarga de toda la lógica. Al iniciar, recibe `escenario_modelo.json` y simula todos los procesos de inicio a fin. El resultado de cada instante de tiempo se guarda como un "fotograma" en `output_modelo.json`.
2. **Reproducción:** Python carga `output_modelo.json` y reproduce la simulación avanzando tick por tick.
3. **Interacciones (E/S):** Si durante la simulación ocurre una interrupción que requiere interacción del usuario (por ejemplo, el Teclado), Python pausará la animación, le pedirá una decisión al usuario (Cancelar o Continuar) y agregará este evento al archivo original `escenario_modelo.json` (en la lista "events").
4. **Recálculo:** Tras esto, Python vuelve a llamar al motor de C++. El backend debe leer el evento, aplicar la decisión en el tick correspondiente, y generar un nuevo `output_modelo.json` actualizado a partir de ese punto.

Estructura Principal del JSON

El archivo JSON de salida debe contener un objeto principal con una lista de `ticks`.

```
{
  "ticks": [
    { /* Fotograma del tick 0 */ },
    { /* Fotograma del tick 1 */ },
    { /* Fotograma del tick 2 */ }
  ]
}
```

Estructura de cada Fotograma (tick)

Cada elemento dentro de `ticks` representa el estado completo del sistema operativo en ese instante de tiempo:

```
{
  "tick": 47,
  "metrics": { ... },
  "cores": [ ... ],
  "process_table": [ ... ],
  "ready_queues": [ ... ],
  "waiting": [ ... ],
  "io_devices": [ ... ],
  "memory": { ... }
}
```

(Nota: Ya no es necesario incluir `timeline` ni `console_logs`. El frontend se encarga de inferir las animaciones y los logs comparando el estado de los procesos entre un tick y el anterior).

1. Tabla de Procesos (process_table)

Esta sección es importante porque el frontend usa estos datos para dibujar el "Inspector de PCB" y actualizar el diagrama de transiciones de estado. Traten de incluir todas las métricas de tiempo y los registros si están disponibles.

```
"process_table": [
{
  "pid": 3,
  "name": "proc",
  "state": "RUNNING",           // NUEVO, READY, RUNNING, WAITING, TERMINATED, ERROR
  "type": "CPU_BOUND",         // SYSTEM, INTERACTIVE, CPU_BOUND, I/O_BOUND
  "priority": 5,
  "burst_time": 20,
  "remaining_time": 15,        // Tiempo que le falta en CPU
  "waiting_time": 5,           // Ticks que ha pasado bloqueado o en la cola de listos
  "arrival_tick": 0,           // Tick en el que se creó el proceso
  "response_time": 2,           // Ticks desde arrival_tick hasta su primera vez en RUNNING
  "finish_time": null,         // Tick donde terminó (solo si el estado es TERMINATED/ERROR)
  "turnaround_time": 0,        // Tiempo total (finish_time - arrival_tick)
  "completion_percent": 25.0,  // (burst_time - remaining) / burst_time * 100

  "pc": 1024,                  // Program Counter numérico
  "pc_hex": "0x0400",          // Program Counter en formato hexadecimal para la UI
  "cpu_id": 0,                 // ID del núcleo asignado (si está en RUNNING)
  "memory_mb": 64,             // Memoria total asignada en MB
  "memory_base_address": 10950, // Dirección física inicial en RAM

  "io_device": null,           // Nombre del dispositivo si está bloqueado (ej. "KEYBOARD")
  "error_code": null,          // Razón del error si está en estado ERROR (ej. "SIGSEGV")

  "registers": {               // Opcional: estado actual de los registros
    "AX": 0, "BX": 42, "CX": 1, "DX": 0
  }
}
]
```

2. Dispositivos E/S (io_devices)

Se utiliza para dibujar los indicadores visuales y barras de progreso de cada componente de hardware.

```

"io_devices": [
  {
    "name": "KEYBOARD",
    "status": "BUSY",           // "IDLE" o "BUSY"
    "queue_length": 1,         // Cantidad de procesos esperando en la cola de este dispositivo
    "current_pid": 14,         // PID del proceso que lo está usando actualmente
    "current_name": "python.exe",
    "progress_percent": 42.0    // Porcentaje de avance de la operación E/S actual
  },
  {
    "name": "DISK",
    "status": "IDLE",
    "queue_length": 0,
    "current_pid": null,
    "current_name": "",
    "progress_percent": 0.0
  }
]

```

3. Manejo de Eventos Asíncronos (Ej. Teclado)

Cuando un proceso hace una petición al Teclado, C++ lo pasará al estado `WAITING` y marcará el `KEYBOARD` como `BUSY` .

Python pausará automáticamente la simulación y mostrará un diálogo al usuario. La decisión del usuario se inyectará al final del archivo `escenario_modelo.json` de la siguiente forma:

```

// Agregado por Python al final de escenario_modelo.json:
"events": [
  {
    "tick": 47,
    "type": "KEYBOARD",
    "pid": 3,
    "action": "CANCEL" // Puede ser "CANCEL" o "CONTINUE"
  }
]

```

Flujo esperado en C++:

Al ejecutarse el motor de C++, debería revisar si existe la llave `"events"` . De ser así, al alcanzar el tick indicado (en este caso particular, el 47), se debe aplicar la acción:

- Si `action` es `"CANCEL"` : Simular una cancelación forzada. El proceso aborta y pasa al estado `"ERROR"` (idealmente agregando un `"error_code": "CANCEL_USR"`), se libera su memoria y el dispositivo queda libre.
- Si `action` es `"CONTINUE"` : Simular que la entrada por teclado finalizó con éxito. El proceso pasa de `"WAITING"` a `"READY"` .

4. Memoria y Paginación (memory)

Este bloque le indica al frontend cómo dibujar el mapa de la RAM.

Para que la barra de memoria muestre los colores correctamente, necesitamos que C++ divida la memoria de cada proceso de usuario en 4 sub-bloques o segmentos: `TEXT` , `DATA` , `HEAP` y `STACK` . Para asignar qué dimensiones deben tener, se podría proponer unas proporciones fijas para brindar tamaños para cada segmento de la memoria del proceso (o quizás aleatorias dentro de ciertos márgenes de números enteros para evitar dificultosa visualización).

```

"memory": {
  "blocks": [
    {
      "start_address": 0,
      "size": 64,
      "is_free": false,
      "process_id": null,
      "segment_type": "OS",
      "label": "SO"
    },
    {
      "start_address": 64,
      "size": 26,
      "is_free": false,
      "process_id": 14,
      "segment_type": "TEXT",      // Código
      "label": "python.exe [TEXT]"
    },
    {
      "start_address": 90,
      "size": 19,
      "is_free": false,
      "process_id": 14,
      "segment_type": "DATA",      // Variables globales
      "label": "python.exe [DATA]"
    },
    {
      "start_address": 109,
      "size": 64,
      "is_free": false,
      "process_id": 14,
      "segment_type": "HEAP",      // Memoria dinámica
      "label": "python.exe [HEAP]"
    },
    {
      "start_address": 173,
      "size": 19,
      "is_free": false,
      "process_id": 14,
      "segment_type": "STACK",      // Pila
      "label": "python.exe [STACK]"
    },
    {
      "start_address": 192,
      "size": 832,
      "is_free": true,
      "process_id": null,
      "segment_type": "FREE",
      "label": "Libre"
    }
  ],
  "stats": {
    "total_mb": 1024,
    "used_mb": 192,
    "free_mb": 832,
    "fragmentation": 12.5,
    "strategy": "FIRST_FIT"
  },
  "mmu_table": {
    "14": {

```

```

        "logical_base": 0,
        "physical_base": 64,
        "size": 128
    }
}
}

```

5. Métricas de Rendimiento (metrics)

```

"metrics": {
    "cpu_utilization": 80.5,
    "throughput": 1.2,
    "avg_turnaround": 34.0,
    "avg_waiting": 12.0,
    "avg_response": 4.5,
    "context_switches": 25,
    "starvation_events": 0,
    "error_rate": 0.5
}

```

6. Núcleos y Colas (cores y ready_queues)

```

"cores": [
    {
        "id": 0,
        "status": "RUNNING",           // Puede ser "IDLE", "RUNNING" o "SWITCHING"
        "current_process": 3,         // PID del proceso en ejecución (null si está inactivo)
        "current_process_name": "proc",
        "switch_overhead_remaining": 0 // Ticks restantes de penalización por cambio de contexto
    }
]

```

```

"ready_queues": [
    {
        "core_id": 0,
        "queue": [
            {
                "pid": 5,
                "name": "brave.exe",
                "type": "CPU_BOUND",
                "priority": 5,
                "waiting_time": 47
            }
        ]
    }
],
"waiting": [
    {
        "pid": 14,
        "name": "python.exe",
        "type": "INTERACTIVE",
        "priority": 5,
        "device": "KEYBOARD",         // Dispositivo por el que está esperando
        "remaining_ticks": 4          // Ticks faltantes para terminar la operación de E/S
    }
]

```